Design and Implementation of a Token-NFT-Liquidity Smart Contract Suite

1st Luca Sforza
sforza.2050030@studenti.uniroma1.it

2nd Roberto Di Rosa
dirosa.2043388@studenti.uniroma1.it,

***Abstract*—Smart contracts allow the creation of applications that run on the blockchain. We aim to present a suite of smart contracts designed to manage NFT (Non-Fungible Token) auctions using a custom token called SapiCoin. We have deployed the auction on the blockchain that utilizes SapiCoin, and to obtain SapiCoin in order to participate in the auction, we have created a Liquidity Pool that enables the exchange of Ether for SapiCoin.**

I. Introduction

From a programming perspective, Smart Contracts can be seen as classes in traditional programming languages such as Java.

To create a Token, one must therefore define a class that represents the Token and manages transactions involving that currency.

Other smart contracts (e.g., an auction) can use Tokens as a means of payment within the contract itself.

However, to avoid binding a smart contract to a specific Token and to promote code reusability, the concept of a Token can be abstracted.

The Ethereum community standardized this abstraction through a document known as ERC-20. [5]

A Token is therefore represented by an Interface (a concept similar to those found in other programming languages).

A smart contract that aims to function as a Token must extend this interface and implement its methods.

The same concept is applied to NFTs, standardized through ERC-721. [2]

In contrast, there is no community standard for auctions, so the design of such smart contracts must be created from scratch.

In addition to token transfers and auctions, our system includes a liquidity pool implemented using Uniswap v3, which allows users to trade between Ether and SapiCoin.

II. Token

ERC-20 is the standard that represents a token on EVM-compatible blockchains.

There is no central authority responsible for issuing these standards; instead, communities such as Ethereum Magicians provide a space to discuss improvements to the Ethereum standard through EIPs [4].

The standard requires that a token is a smart contract implementing a specific set of methods.

```
contract SapiCoin is ERC20 {
    /* implementation */
}
```

Fig. 1. Polymorphism on Solidity

A. Polymorphism in the EVM

Achieving this requires polymorphism.

The Ethereum Virtual Machine does not provide instructions for handling abstract classes, but the VM itself is polymorphic.

When a method of a smart contract is invoked through a transaction, the memory address of the method is accessed by computing the hash of the function signature.

If the invoked method does not exist, a default fallback function is executed (which can be overridden).

Thanks to this behavior of the EVM, it is possible to implement any standard.

If we want to implement a contract that manages an NFT auction, it will call the functions whose signature hashes match those defined by the ERC-20 standard.

B. Design of the standard ERC-20

A smart contract can be seen as a class in other programming languages, and can therefore be represented as a UML class.

For the development of blockchain smart contracts, a structured approach based on the principles of the Model Driven Architecture can be beneficial and facilitate the implementation of smart contracts. In combination with Unified Modeling Language (UML) Class and State machine diagrams, allows the smart contract structure and behavior logic to be modeled in several abstraction layers. [3]

So we can model the ERC-20 as shown in Fig 2.

The ERC-20 standard requires a token to implement the following methods:

1) **balanceOf(address owner)** returns the amount of tokens owned by a given address.
2) **transfer(address to, uint256 value)** allows sending tokens to a specific address (the `to` parameter).
3) **approve(address spender, uint256 value)** allows an address (the `spender`) to spend a certain amount of tokens owned by the address that submits the transaction.

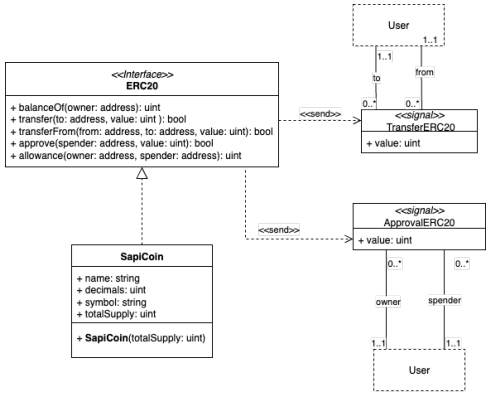

Fig. 2. UML Class Diagram of a Token.

```
    event Transfer(address indexed _from, address
    indexed _to, uint256 _value);
    event Approval(address indexed _owner, address
    indexed _spender, uint256 _value);
```

Fig. 3. Events on Solidity

4) **transferFrom(address from, address to, uint256 value)** allows transferring a specified amount of tokens (`value`) from one address (the token owner, not necessarily the transaction sender) to another address. Obviously, it must not be possible for someone without permission to spend our tokens; therefore, the sender must have been granted approval (via the `approve` method).
5) **allowance(address owner, address spender)** returns how many tokens an `owner` has approved for a given `spender`.

The entire approval mechanism is very useful from a programming perspective.

Users can call `approve` on a given smart contract for a specified amount of tokens.

In this way, a smart contract can use our tokens to provide a service.

ERC-20 also defines a set of events that a token should emit whenever the contract's internal state changes, so that the operations executed within a smart contract can be tracked and displayed by front-end applications such as Etherscan.

C. Events

Figure 2 describes the various events emitted asynchronously by ERC-20, specifically:

1) **Transfer**, which tracks token transfers between users.
2) **Approval**, which tracks approvals from an owner to a spender.

Solidity provides a construct to represent these events, as shown in Fig. 3.

D. SapiCoin Implementation

To implement a specific token, in our case SapiCoin, there is no additional logic beyond being a token. If we wanted to implement a stablecoin or a wrapped coin, we would need to add methods to manage the exchange of the token with the real-world currency it represents (for example, USDT or WETH).

The implementation of the other standard token functionalities is easily available online (in some cases, they are embedded in Solidity development environments).

The only aspects modified are the token's "vanity" features, namely the token's name, the number of decimals the token uses (in our case 18 decimals), and the token symbol (SAC).

With 18 decimals, the smallest amount of SapiCoin that can be obtained is $1 \cdot 10^{-18}$.

However, being just a vanity feature, at a low level we only have access to a 256-bit unsigned integer.

For example, 4 SapiCoin in memory is stored as $4 \cdot 10^{18}$.

III. NFTs

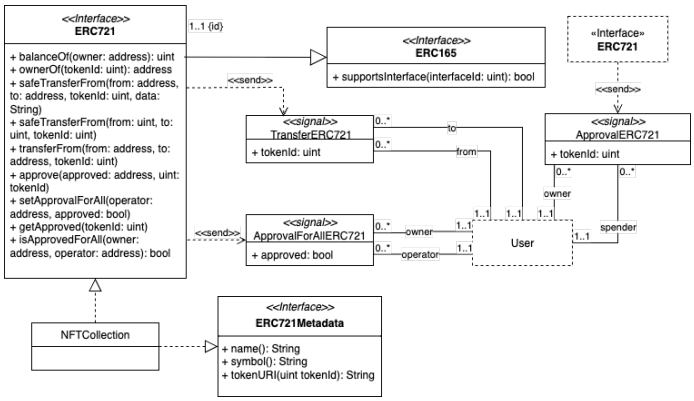

Fig. 4. UML Class Diagram of a NFT.

According to the standard [2] , every ERC-721 compliant contract must implement the ERC721 and ERC165 interfaces, as shown in Figure 4.

A. ERC-165 Standard

The ERC-165 standard defines a mechanism to declare and verify whether a contract implements a specific interface. In other words, it allows other contracts to query a contract and determine if it supports a certain functionality (e.g., ERC-721, ERC-20, etc.).

supportsInterface(bytes4 interfaceID): allows checking whether the contract implements the interface identified by `interfaceID`. Returns `true` if the interface is supported, and `false` otherwise.

According to the standard, the supportsInterface function must return true if and only if the provided interfaceID matches the XOR of the function selectors that define the interface.

However, Solidity provides a built-in mechanism to obtain this value directly through the language syntax itself, resulting in a precomputed and therefore more gas-efficient representation (see Figure 5).

```
contract Taxpayer is IERC165, ITaxpayer {

    mapping(bytes4 => bool) internal
supportedInterfaces;

    constructor() {
        supportedInterfaces[
            type(IERC165).interfaceId
        ] = true;
        supportedInterfaces[
            type(ITaxpayer).interfaceId
        ] = true;
    }
    function supportsInterface(
        bytes4 interfaceId
    ) external view returns (bool) {
        return supportedInterfaces[interfaceId];
    }
}
```

Fig. 5. The most gas efficient implementation of ERC-165

B. ERC-721 Standard

The ERC-721 interface defines the standard for NFTs. Each token is unique and distinct, unlike ERC-20 tokens which are fungible (all identical). The functions to implement are:

1) **balanceOf(address owner)**: returns the number of NFTs owned by a given address.
2) **ownerOf(uint256 tokenId)**: returns the address of the owner of the token identified by `tokenId`.
3) **safeTransferFrom(address from, address to, uint256 tokenId, bytes data)**: safely transfers ownership of an NFT from one address to another. If the recipient is a contract, the function `onERC721Received` is called to verify that it can receive NFTs. For the purposes of this project, we ignored this case, and the `auction` contract does not implement `onERC721Received`. It throws an error if:
   - `msg.sender` is not the owner or an authorized operator;
   - `from` is not the current owner;
   - `to` is the zero address;
   - the token does not exist.
4) **safeTransferFrom(address from, address to, uint256 tokenId)**: identical to the previous function but without the additional `data` parameter (set to an empty string).
5) **transferFrom(address from, address to, uint256 tokenId)**: transfers ownership of an NFT without performing additional checks on the recipient's ability to handle it. It is the caller's responsibility to ensure that `to` can handle NFTs. Throws errors under the same conditions as `safeTransferFrom`.
6) **approve(address approved, uint256 tokenId)**: allows the owner to approve another address (`approved`) to transfer the specified token (`tokenId`). Passing the zero address removes any existing approval.
7) **setApprovalForAll(address operator, bool approved)**: allows a user to enable or disable an operator that can manage all of their NFTs. Emits the `ApprovalForAll` event and allows multiple operators per owner.
8) **getApproved(uint256 tokenId)**: returns the address approved to manage a specific token. Returns the zero address if no address is approved. Throws an error if the token is invalid.
9) **isApprovedForAll(address owner, address operator)**: checks if an operator has been approved to manage all tokens of a given owner. Returns `true` if authorized, `false` otherwise.

Like ERC-20, contracts implementing ERC-721 should emit the following events:

1) **Transfer(address indexed from, address indexed to, uint256 indexed tokenId)**: emitted whenever ownership of an NFT changes, whether by transfer, creation (`from == 0`), or destruction (`to == 0`). When a transfer occurs, any approved address for that token is automatically cleared.
2) **Approval(address indexed owner, address indexed approved, uint256 indexed tokenId)**: emitted when an approved address is set or removed for a specific NFT. The zero address indicates that no approved address exists for that token.
3) **ApprovalForAll(address indexed owner, address indexed operator, bool approved)**: emitted when an owner enables or disables an operator to manage all their NFTs. The operator can then perform transfers or approvals on behalf of the owner.

C. Optional Metadata and Vanity

Extending the `ERC721Metadata` interface is not mandatory for the ERC-721 standard; it is used solely to add vanity features to NFTs.

Through this interface, we can obtain the name of the NFT collection, the collection symbol, and, via the **tokenURI(uint256 tokenId)** method, the URL associated with the NFT.

This link can point to the image that the NFT represents or to other related information.

The functionalities included depend heavily on what we want to incorporate into our NFT, but the underlying architecture of NFTs remains the same.

Indeed, for the ERC-721 standard, just like for ERC-20, there are many reusable implementations available for our collection.

For the purpose of this project, we created an NFT for the image in Figure 6.

IV. Auction

There is no standard for NFT auctions, so creating one requires using other strategies.

For example, one can use an auction system with contracts already deployed on the blockchain and create an auction through that system. This way, every user of the system can access your auction, for instance via OpenSea.

Fig. 6. Image associated with the NFT we minted

Alternatively, one can manually create the auction smart contract and deploy it on the blockchain.

With the second approach, the drawback is that there is no front-end application to interact with the service. However, it is possible to publish the contract on Etherscan so that, by connecting a wallet (such as MetaMask), users can interact with the smart contract directly through a web browser.

Unfortunately, the main NFT marketplaces have removed their support for testnet blockchains, so we opted to implement it from scratch.

A. Design of Auction

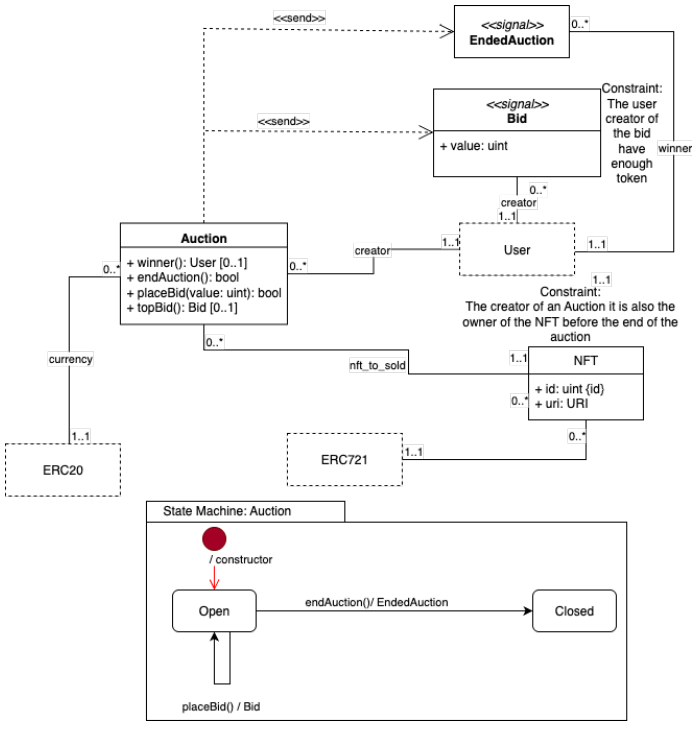

Fig. 7. UML of a Auction

The goal of the auction we designed is to create a smart contract that is reusable with any type of token and any NFT collection.

Here is a description of the auction methods:

1) **Winner**(): Returns the address that has placed the highest bid at the moment the method is called. The address is zero if no bids have been placed yet.
2) **topBid**(): Returns the address that has placed the highest bid along with the bid amount at the moment the method is called. Both the address and the bid value are zero if no bids have been placed yet.
3) **placeBid(uint256 value)**: Allows placing a bid in the auction. To execute this method, the auction must first be approved to transfer your tokens. When `placeBid` is executed, the auction takes the tokens specified in the `value` field and transfers them to itself. It is recommended to approve the auction to transfer an unlimited number of tokens so that each bid placement does not require two transactions. If a bid has already been placed, the value is added to the existing bid.
4) **endAuction**(): Allows only the creator of the auction to close it. Once the auction is closed, no further bids can be placed. The winner receives the NFT (before calling this method, the auction creator must approve the token transfer), and the creator receives the tokens. All other bidders are refunded.

B. Events

Events are emitted for every change in the smart contract's state, as shown in Figure 7.

Specifically, the events are:

1) **Bid(uint256 value, address indexed creator)**: Emitted every time a bid is placed.
2) **endedAuction(address indexed winner)**: Emitted when the auction is closed.

V. Implementation Details

Solidity, unlike traditional programming languages, introduces additional constraints related to gas consumption, security, and transaction management.

In particular, Solidity development environments allow direct interaction with the blockchain, enabling developers to perform transactions, deploy smart contracts, and publish contract source code on services such as Etherscan.

One of the most significant constraints imposed by Solidity is that, once a contract is deployed on the blockchain, it cannot be modified. Since deploying smart contracts may incur substantial costs, ensuring the correctness of the code becomes essential. Consequently, testing—or even formal verification of the code, understood as proving that the system's invariants are satisfied—is a critical step.

To verify correctness, one may employ logic-based approaches (First-Order Logic, Hoare Logic) to formally prove properties of the code. Alternatively, fuzz testing can be used: developers define the invariants and repeatedly invoke contract methods with random inputs until a violation of those invariants is detected.

1) State of Art: There are many development environments for Solidity. We chose Foundry. Another widely used option is OpenZeppelin.

A. Foundry

Foundry provides a complete ecosystem for working with EVM-compatible blockchains.

It consists of three main utilities:

1) `forge`, which enables compilation, testing, debugging, deployment, and verification of smart contracts.
2) `anvil`, which allows the instantiation of a local development node by forking the selected blockchain.
3) `cast`, a versatile command-line tool for interacting with on-chain applications.

`forge` is the primary tool used when developing in Solidity.

forge initializes a new project (structured as a directory), where smart contracts are placed in the `src` folder.

Once the code is written, the smart contract can be deployed as follows:

```
$ forge create src/sapicoin.sol:SapiCoin  -rpc
    -url https://ethereum-sepolia-rpc.
    publicnode.com
-private-key $(cat path/to/pk.txt)
-broadcast -constructor-args $@ # constructor
    arguments
```

By providing the private key, forge can sign transactions on the selected blockchain through the RPC endpoint, which serves as the interface enabling external tools to communicate with a network node (sending transactions, reading state, querying blocks).

The output of this command is the address of the deployed smart contract.

There are two main approaches for interacting with a deployed smart contract:

1) Programmatically, by binding Solidity methods in Rust (or in JavaScript in other development environments).
2) By verifying contract ownership on external platforms (such as Etherscan) and interacting directly from a web browser via a connected wallet.

The command for verifying a smart contract is:

```
$ forge verify-contract -rpc-url https://
    ethereum-sepolia-rpc.publicnode.com
-etherscan-api-key $(cat path/to/api_key.txt)
$(echo $ADDRESS)
src/sapicoin.sol:SapiCoin
```

Registration on Etherscan is required to obtain an API key.

Verification sends the Solidity compiler configuration to Etherscan, which then recompiles the code and checks that the resulting bytecode matches the on-chain contract.

This process allows anyone familiar with Solidity to confirm that the contract is not malicious, and Etherscan subsequently enables public interaction with it.

VI. Liquidity Pool

As the creators of the SapiCoin token, we initially hold the entire token supply. Consequently, without additional mechanisms, the only way to distribute SapiCoin to users would be to send them manually to those who request them. This approach is not scalable and does not allow users to obtain tokens autonomously.

To automate distribution and, more importantly, to create a real market between SapiCoin and Ether, we introduced an Automated Market Maker (AMM). AMMs are protocols that allow users to swap tokens without the need for a direct counterparty.

For this project, we used Uniswap v3 [1] , one of the most advanced and efficient AMMs. Uniswap v3 allows the creation of liquidity pools between SapiCoin and Ether. By depositing an initial amount of both tokens, an initial market price for SapiCoin is established. From that point onward, anyone can buy or sell SapiCoin against Ether, and the protocol automatically updates the price based on the AMM formula and the available liquidity.

In fiat currencies, central banks (e.g., the European Central Bank) determine the exchange rate. In DeFi currencies, this is not possible because we operate in a distributed system.

Uniswap v3 uses the Constant Product Formula to determine the exchange rate:

$$x \cdot y = k \tag{1}$$

Here, x and y are the amounts of the two tokens available for swapping, and k is the invariant of the system. We can freely change the values of x and y as long as k remains constant.

Therefore, the exchange rate of token x in exchange for y is $\frac{x}{y}$.

Those who provide liquidity are called investors, creating positions in the liquidity pool. Investors provide liquidity to earn a return. This return comes from fees. Depending on the amount of liquidity contributed to the pool, each investor receives a percentage of the fees collected.

The liquidity within a liquidity pool is calculated as:

$$L = \sqrt{k} = \sqrt{x \cdot y} \tag{2}$$

For each investor, the amount of liquidity invested in their positions is recorded. When an investor contributes, their position is represented by an NFT.

This allows investors to sell or trade their liquidity positions.

VII. Conclusion

We have designed and implemented a complete suite of smart contracts that interact through well-established Ethereum standards. The Token contract follows the ERC-20 specification and provides a fungible currency (SapiCoin) used as the medium of exchange within the system. The NFT contract adheres to the ERC-721 standard and defines unique assets that can be auctioned. Finally, the Auction and Liquidity

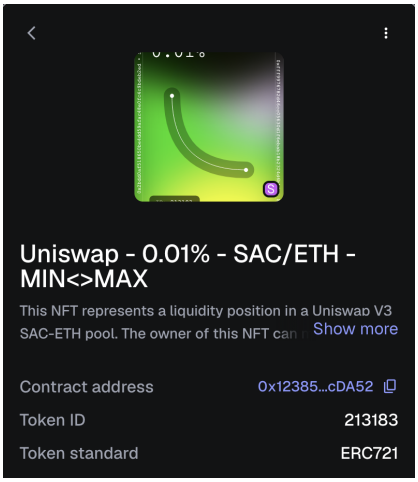

Fig. 8. NFT of a liquidity position in Uniswap v3.

Pool contracts coordinate token transfers, NFT ownership changes, and Ether-SapiCoin conversion.

The architecture demonstrates how interoperability on Ethereum naturally emerges from UML-style interface-based design and the polymorphic behavior of the EVM. Each component is independent yet composable, enabling reuse, modular upgrades, and clear separation of concerns. The Liquidity Pool offers an automated mechanism to acquire SapiCoin, removing the need for manual token distribution, while the Auction contract provides a programmable marketplace for NFTs.

Overall, the system shows how standardized interfaces (ERC-20, ERC-165, ERC-721) combined with custom logic make it possible to build decentralized applications where multiple contracts cooperate reliably without central authority.

REFERENCES

[1] Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, and Dan Robinson. Uniswap v3 core, March 2021. Whitepaper / Technical Report.

[2] William Entriken, Dieter Shirley, Jacob Evans, and Nastassia Sachs. Erc-721: Non-fungible token standard. Ethereum Improvement Proposal, January 2018. Standard interface for non-fungible tokens on Ethereum.

[3] Mantas Jurgelaitis, Lina Čeponienė, and Rita Butkienė. Solidity code generation from uml state machines in model-driven smart contract development. *Department of Information Systems, Faculty of Informatics, Kaunas University of Technology*, 2022.

[4] Fellowship of Ethereum Magicians. Fellowship of ethereum magicians, s.d. Accessed: 2025-11-19.

[5] Fabian Vogelsteller and Vitalik Buterin. Erc-20: Token standard. Ethereum Improvement Proposal, November 2015. Standard interface for fungible tokens on Ethereum.